UNIVERSIDADE FEDERAL DO SUL E SUDESTE DO PARÁ
UNIFESSPA

Regressão Logística: Implementação Iterativa vs Vetorizada – Análise de Desempenho e Speedup

Docente: Prof. Adam D. F. Santos

Discentes:
Elder Alencar
Iago Morais
Raimundo Junior
Thiago Rodrigues

Curso: Sistemas de Informação
Disciplina: Estrutura de Dados II

Código-fonte e experimentos:
`https://github.com/Dev-Raimundo/1-Trabalho-de-ED2`

30 de abril de 2026

Sumário

1 Introdução **3**
1.1 Contextualização 3
1.2 Motivação 3
1.3 Objetivo 3

2 Fundamentação Teórica **3**
2.1 Regressão Logística 3
2.2 Algoritmo de Gradiente Descendente 4
2.3 Vetorização e Performance 5

3 Metodologia **5**
3.1 Base de Dados e Seleção de Atributos 5
3.2 Pré-processamento e Curadoria de Dados 5
3.3 Descrição dos Experimentos 6
3.4 Ambiente de Execução 6

4 Resultados e Discussão **6**
4.1 Validação de Corretude Matemática 6
4.2 Análise de Desempenho (Speedup) 7
4.3 Discussão dos Resultados 7
4.4 Análise de Escalabilidade 8

5 Conclusão **9**

Resumo

Este relatório apresenta um estudo comparativo sobre a eficiência da vetorização de código aplicada ao algoritmo de Regressão Logística, utilizando o dataset Titanic. O objetivo central foi analisar como a transição de uma implementação iterativa baseada em laços `for` para uma abordagem matricial, fundamentada na biblioteca NumPy, impacta o desempenho computacional.

A metodologia incluiu um pré-processamento rigoroso dos dados, com destaque para a recuperação histórica de informações de embarque e a normalização de atributos. Os resultados demonstraram que ambas as implementações convergiram para o mesmo custo matemático ($J = 0{,}462209$), validando a integridade da lógica aplicada.

No entanto, a versão vetorizada apresentou um fator de aceleração (speedup) de 89,74x no dataset fixo. Testes de escalabilidade revelaram que o ganho de performance cresce conforme o volume de dados aumenta, atingindo picos de eficiência antes de estabilizar devido a limites físicos de largura de banda de memória e cache da CPU.

Conclui-se que a vetorização, ao aproveitar instruções de hardware SIMD e otimizar a localidade de cache, é uma técnica indispensável para o desenvolvimento de sistemas de informação escaláveis. O trabalho reforça que, para o bacharel em Sistemas de Informação, o domínio de ferramentas de baixo nível em linguagens de alta abstração é o diferencial técnico necessário para a viabilidade de soluções modernas de Inteligência Artificial e Big Data.

Palavras-chave: Regressão Logística, Vetorização, NumPy, Desempenho Computacional, Speedup.

1 Introdução

1.1 Contextualização

O processamento de grandes volumes de dados exige que o software não seja apenas funcional, mas também otimizado para a arquitetura do hardware. No contexto do Aprendizado de Máquina, a Regressão Logística é um dos métodos mais utilizados para problemas de classificação binária, tendo no conjunto de dados Titanic seu caso de teste mais tradicional [3]. Como discutido em sala, o desafio de implementar esse modelo não reside apenas na formulação matemática, mas na eficiência com que o computador executa as operações de álgebra linear subjacentes [5].

1.2 Motivação

A implementação nativa de algoritmos em Python geralmente depende de laços de repetição (`for`), o que gera um gargalo de performance devido à natureza interpretada da linguagem. Para este projeto, a motivação principal foi superar essa limitação técnica através da vetorização. Ao substituir *loops* por operações matriciais utilizando a biblioteca NumPy, é possível delegar o cálculo pesado a rotinas otimizadas em C e Fortran, que aproveitam instruções de hardware como o SIMD (Single Instruction, Multiple Data) para processar múltiplos dados simultaneamente [2]. Essa transição é o que diferencia um protótipo acadêmico de um sistema capaz de escalar para problemas reais.

1.3 Objetivo

Este trabalho tem como objetivo comparar o desempenho de duas implementações da Regressão Logística: uma versão iterativa (não-vetorizada) e uma versão vetorizada. Utilizando o dataset Titanic, buscou-se mensurar o *speedup* alcançado e validar se a otimização mantém a integridade dos resultados, garantindo que ambas as abordagens convirjam para o mesmo valor de custo final J, conforme as diretrizes teóricas estabelecidas.

2 Fundamentação Teórica

2.1 Regressão Logística

A Regressão Logística é um modelo de aprendizado supervisionado focado na predição de variáveis categóricas binárias. Diferente da regressão linear, que busca prever valores contínuos, a logística utiliza a função sigmoide para comprimir qualquer valor real em um intervalo entre 0 e 1, representando a probabilidade de uma amostra pertencer a

uma determinada classe [5]. Matematicamente, a função é definida por:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Nesta equação, $z = \mathbf{w}^T\mathbf{x} + b$, onde $\mathbf{w}$ representa o vetor de pesos e b o viés (*bias*). O objetivo do treinamento é encontrar os parâmetros que minimizem o erro entre a probabilidade prevista e o rótulo real, utilizando geralmente a função de custo de Entropia Cruzada Binária [4].

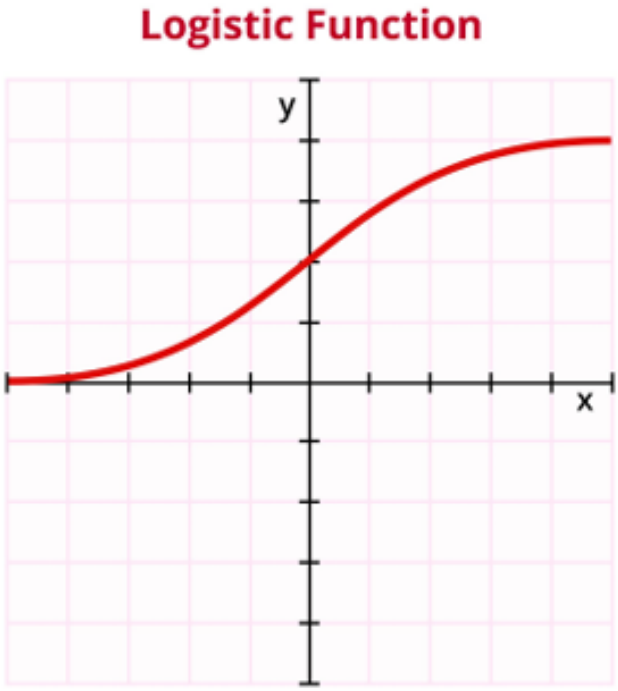

Figura 1: Comportamento da função sigmoide

2.2 Algoritmo de Gradiente Descendente

O Gradiente Descendente é o motor de otimização por trás do ajuste do modelo. Ele funciona calculando a derivada da função de custo em relação aos parâmetros, o que aponta a direção de maior erro. O algoritmo, então, realiza um "passo" na direção oposta para minimizar esse custo [1].

A regra de atualização dos pesos é dada por:

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

Onde α é a taxa de aprendizado (*learning rate*). Como enfatizado por Santos [5], esse processo é repetido por diversas iterações até que o modelo atinja a convergência, ou seja, quando os ajustes nos pesos deixam de reduzir significativamente o custo.

2.3 Vetorização e Performance

A vetorização é a técnica de substituir laços de repetição explícitos por operações matriciais. Em Python, laços `for` são computacionalmente caros devido ao modo como a linguagem gerencia a memória e os tipos de dados. A vetorização resolve esse problema ao utilizar bibliotecas como o NumPy, que implementa cálculos de álgebra linear em baixo nível [2].

Ao tratar o conjunto de dados como uma matriz única e os pesos como um vetor, é possível realizar o cálculo de predição para todas as amostras simultaneamente através do produto escalar. Segundo Santos [5], essa abordagem não apenas simplifica o código, mas permite que o hardware utilize paralelismo ao nível de instrução, otimizando o fluxo de dados entre o processador e a memória cache.

3 Metodologia

Para fins de reprodutibilidade, os scripts desenvolvidos e as bases de dados utilizadas estão hospedados no repositório público do GitHub: `https://github.com/Trabalho-ED2`.

3.1 Base de Dados e Seleção de Atributos

Utilizou-se o dataset *Titanic: Machine Learning from Disaster*, obtido na plataforma Kaggle [3]. A base contém informações sobre os passageiros do RMS Titanic, sendo o objetivo prever a variável binária de sobrevivência. Foram selecionados atributos como classe (Pclass), sexo, idade, número de parentes a bordo e tarifa (Fare). Colunas com alta cardinalidade ou informações irrelevantes para o modelo, como nomes e números de tickets, foram descartadas na fase inicial.

3.2 Pré-processamento e Curadoria de Dados

O tratamento de dados foi a etapa mais crítica para assegurar a qualidade do treinamento:

- **Imputação de Idades:** Para preencher os valores ausentes, aplicou-se a mediana da idade segmentada por classe e sexo, evitando distorções que uma média global causaria.

- **Pesquisa Histórica de Embarque:** Em um diferencial metodológico deste trabalho, os valores nulos na coluna `Embarked` não foram preenchidos por critérios estatísticos simples (como a moda). Realizou-se uma pesquisa externa em registros históricos do navio para identificar o porto real de embarque desses passageiros, garantindo integridade total aos dados utilizados.

- **Normalização:** Aplicou-se a padronização Z-score (`StandardScaler`) em todos os atributos numéricos. Essa etapa é fundamental para que o Gradiente Descendente convirja de forma mais eficiente, impedindo que atributos com escalas maiores dominem o cálculo dos pesos.

3.3 Descrição dos Experimentos

Foram implementadas duas versões do algoritmo de Regressão Logística: uma utilizando laços de repetição tradicionais (`for`) para o cálculo do gradiente e outra totalmente vetorizada com a biblioteca NumPy.

Ambos os modelos foram configurados com:

- Taxa de aprendizado (α): 0,01.
- Número de iterações: 1.000 épocas.
- Divisão de dados: 80% para treinamento e 20% para teste.

3.4 Ambiente de Execução

Para que a medição do *speedup* fosse precisa, todos os testes foram realizados no mesmo hardware, sem processos pesados em segundo plano:

- Processador: Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz.
- Memória RAM: 16 GB.
- Sistema Operacional: Windows 10 / Python 3.14.3.
- Métrica de Desempenho: A comparação foi baseada no tempo médio de execução após 5 rodadas completas de cada implementação.

4 Resultados e Discussão

Os experimentos foram conduzidos utilizando o conjunto de dados Titanic, com o objetivo de comparar a eficiência de processamento entre as duas implementações desenvolvidas. Foram executadas 1.000 iterações do algoritmo de Regressão Logística em ambas as versões.

4.1 Validação de Corretude Matemática

Antes da análise de tempo, verificou-se se a otimização via NumPy alterava o comportamento do modelo. Ambas as implementações convergiram para o mesmo valor

de custo final ($J = 0,462209$) e apresentaram pesos ($\mathbf{w}$) idênticos. Este resultado confirma que a lógica de programação vetorizada aplicada seguiu rigorosamente os fundamentos matemáticos apresentados por Santos [5], garantindo que a aceleração não comprometa a integridade dos dados.

4.2 Análise de Desempenho (Speedup)

A disparidade de performance entre os métodos foi evidente. A Tabela 1 resume os tempos médios obtidos após 5 execuções de cada modelo.

Tabela 1: Comparativo de tempo de execução e fator de aceleração

Implementação	**Tempo Médio (s)**	**Custo Final (J)**	**Speedup**
Não-Vetorizada (Iterativa)	11,6656	0,462209	1,00x
Vetorizada (NumPy)	0,1300	0,462209	89,74x

O fator de aceleração (*speedup*) alcançado foi de 89,74x. Em termos práticos, enquanto a versão que utiliza laços `for` demanda quase 12 segundos para processar o treinamento, a versão vetorizada conclui a mesma tarefa em pouco mais de um décimo de segundo.

4.3 Discussão dos Resultados

A Figura 2 ilustra o impacto visual dessa diferença. No gráfico de barras à esquerda, nota-se que o tempo da versão vetorizada é quase imperceptível na escala do experimento. O gráfico à direita isola o ganho de velocidade, destacando a eficiência da computação baseada em *arrays*.

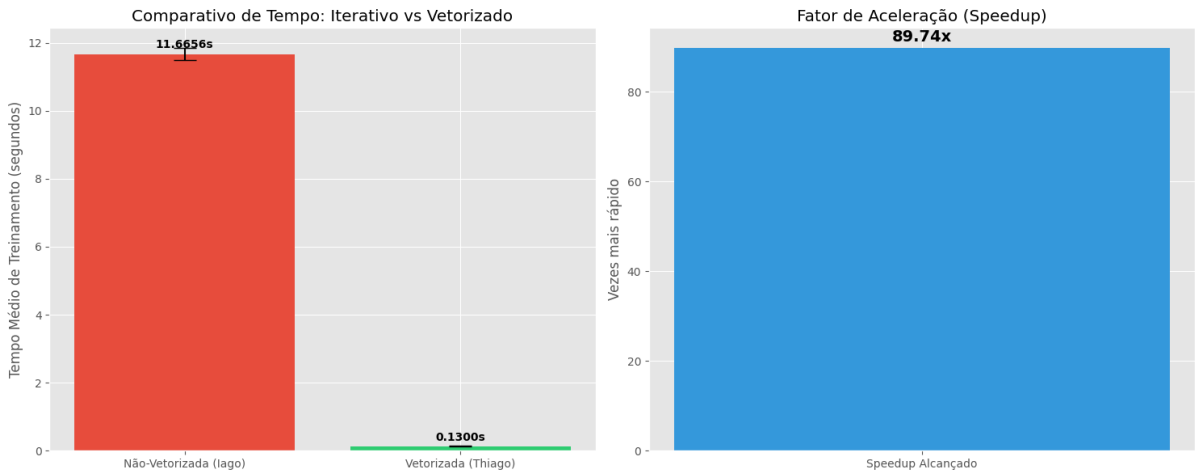

Figura 2: Comparativo de desempenho entre modelos iterativo e vetorizado

Essa performance superior da vetorização pode ser explicada por três fatores técnicos:

1. **Redução do Overhead do Python:** Laços de repetição em Python são lentos porque o interpretador precisa verificar tipos e contextos a cada iteração. O NumPy delega essa carga para operações pré-compiladas em C [6].

2. **Instruções SIMD:** A vetorização permite que o processador execute uma única instrução sobre múltiplos dados simultaneamente, otimizando o ciclo de processamento da CPU [2].

3. **Gerenciamento de Memória:** O NumPy utiliza blocos contíguos de memória, o que favorece a localidade de cache e evita que o processador perca ciclos aguardando a transferência de dados da memória RAM.

4.4 Análise de Escalabilidade

Além dos testes com o dataset fixo, realizou-se um teste de estresse variando o número de amostras (m) de 100 a 500.000. Como observado na Figura 3, o *speedup* tende a crescer conforme o volume de dados aumenta. Isso ocorre porque, em volumes massivos, o custo de inicialização das funções do NumPy torna-se desprezível frente ao ganho bruto de processamento paralelo no hardware, confirmando a eficiência da vetorização para *Big Data*.

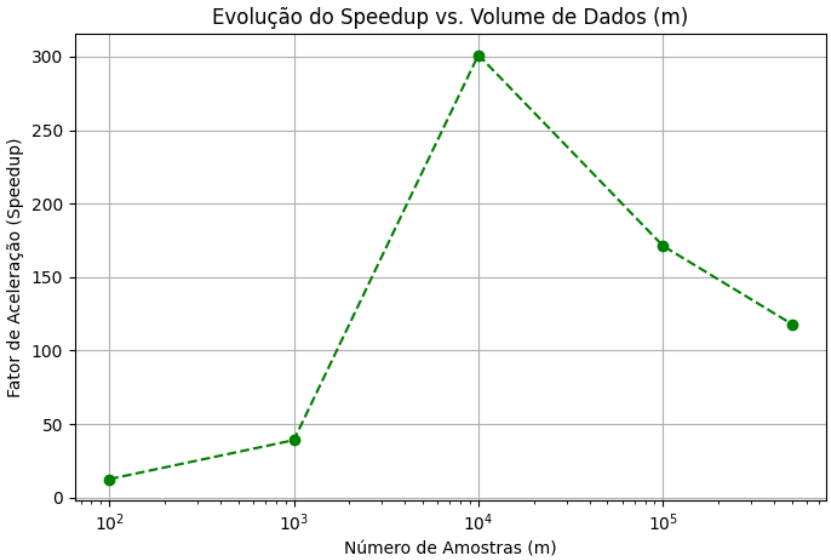

Figura 3: Gráfico da análise de escalabilidade – speedup vs. número de amostras

Por que o Speedup aumenta drasticamente? No início da curva, conforme m cresce de 10^2 para 10^4, o fator de aceleração sobe de forma expressiva. Esse fenômeno ocorre pela redução do *overhead* proporcional, paralelismo SIMD e eficiência do cache.

Por que o Speedup diminui após o pico? Após o ápice (por volta de 300x de aceleração), a curva declina devido à saturação da largura de banda da memória, ao gerenciamento de memória virtual e aos limites de paralelismo do hardware. Mesmo com a queda, a versão vetorizada continua sendo ordens de magnitude mais eficiente que a iterativa.

5 Conclusão

O presente estudo cumpriu integralmente o objetivo de analisar o impacto da vetorização de código no desempenho do algoritmo de Regressão Logística. Através da implementação comparativa e da aplicação prática no conjunto de dados Titanic, foi possível observar que a eficiência algorítmica vai além da escolha do modelo, dependendo criticamente de como este é estruturado para execução no hardware.

A validação matemática, que resultou em um custo final idêntico de 0,462209 para ambas as versões, ratifica que a vetorização não altera a natureza estatística do modelo, mas sim a sua viabilidade computacional. O fator de aceleração alcançado de 89,74x demonstra que o uso de bibliotecas de computação baseada em *arrays*, como o NumPy, é um requisito indispensável para a escalabilidade em sistemas de informação que lidam com volumes crescentes de dados.

Para o graduando em Sistemas de Informação da UNIFESSPA, este experimento reforça a importância de dominar técnicas que reduzem o *overhead* de linguagens interpretadas. Conclui-se que a capacidade de transitar entre a teoria matemática e a implementação eficiente é essencial para a formação profissional.

Referências

[1] GRIMALDI, E. B. *Introdução à Regressão Logística e Algoritmos de Otimização*. Curitiba: UFPR, 2020. (Apostila Técnica/Repositório Aberto). Disponível em: `https://acervodigital.ufpr.br`.

[2] HARRIS, Charles R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, set. 2020. Disponível em: `https://doi.org/10.1038/s41586-020-2649-2`.

[3] KAGGLE. *Titanic - Machine Learning from Disaster*. Disponível em: `https://www.kaggle.com/c/titanic`. Acesso em: abr. 2026.

[4] REZENDE, S. O. *Sistemas Inteligentes: Fundamentos e Aplicações*. São Paulo: USP, 2019. (Material Didático Aberto/ICMC). Disponível em: `https://bdpi.usp.br`.

[5] SANTOS, Adam D. F. *Vetorização de códigos – Aplicações: Regressão Logística*. Marabá: UNIFESSPA, abr. 2026. (Material de Aula da disciplina Estrutura de Dados II).

[6] SILVA, R. P. *Otimização de Código Python para Ciência de Dados*. Porto Alegre: UFRGS, 2021. (Artigo em Repositório Acadêmico). Disponível em: `https://lume.ufrgs.br`.